

TABLE OF CONTENTS

<u>Sr.No</u>	<u>Title</u>	<u>Page No</u>
1	Title Page	1
2	Certificate of Approval	2
3	Acknowledgement	3
4	Internship Certificate	4 - 6
5	Table of Content	7
6	Executive Summary	8
	• Introduction of the Report	9 - 13
	• Overview of the Internship providing Organization	14 - 16
	• About the Training	17 - 30
	• Conclusion	31 - 32

EXECUTIVE SUMMARY

The internship under the ADM Java Full Stack – React (Stage 2 Plus) track at Cognizant Technology Solutions, Chennai, provided a comprehensive understanding of modern full-stack application development, backend engineering, microservices architecture, and DevOps practices. The program was designed to bridge the gap between academic knowledge and industry requirements by offering hands-on exposure to real-world technologies, frameworks, and delivery-focused agile workflows.

The training covered key domains such as Core Java programming and Object-Oriented Programming (OOP), Data Structures and Algorithms, Design Patterns and Principles, advanced backend technologies including Spring Core, Spring Boot 3, Spring Data JPA with Hibernate, and Spring REST APIs. Frontend development was extensively covered through Node.js, TypeScript, Bootstrap 5, and React (v18). Additionally, the internship introduced microservices architecture using Spring Cloud, containerization with Docker, AI-assisted development using GitHub Copilot, code quality enforcement using SonarQube, and cloud deployment using Amazon Web Services (AWS).

The DevLed Skilling methodology adopted by Cognizant ensured that the training simulated a real-world software delivery environment. Participants worked in agile pods, attended Scrum ceremonies, followed sprint cycles, and progressed through a capstone Integrated Development Project (IDP). Continuous assessments, hands-on coding exercises, Interim and Final Evaluations, and a HackerRank Final Assessment validated technical proficiency, problem-solving skills, and project readiness.

Overall, this internship strengthened my ability to design, develop, test, and deploy scalable full-stack applications, while providing valuable insights into modern software development life cycles and enterprise delivery practices. This report presents a detailed overview of the learning journey, technologies explored, and outcomes achieved during the internship.

1 Introduction of the Report

This section describes the academic and professional backdrop of my internship with Cognizant Technology Solutions India Private Limited, under the ADM Java Full Stack – React (Stage 2 Plus) track of the GenC Programmer Analyst Trainee program. It discusses the objectives, relevance, training progression, tools used, and evaluation techniques. The internship was part of the academic program and was designed to provide structured, industry-relevant learning experiences in modern software development and full-stack application engineering.

The internship commenced on February 24, 2026, with a Bootcamp and Induction phase running from February 24 to March 4, 2026, where company culture, policies, code of conduct, professional ethics, and initial workplace setup were covered. From March 5, 2026 onwards, the structured Technical Domain Training began as part of the Java Full Stack React track, organised in five agile sprints over a 14.24-week immersive program.

The training began with an introduction to programming fundamentals through Core Java refresher, Object-Oriented Programming, Data Structures and Algorithms, and Design Patterns, and progressed through organized training in Spring Core, Spring Boot 3, Spring Data JPA, Hibernate, Spring REST, JUnit, Mockito, Microservices with Spring Cloud, Docker, Node.js, TypeScript, Bootstrap 5, React (v18), SonarQube, and AWS Cloud and DevOps. The training also included regular assessments, hands-on coding exercises, agile ceremonies, and a capstone Integrated Development Project.

Cognizant is a global leader in IT services, consulting, and digital transformation, helping organizations build scalable, secure, and high-performance software solutions across industries such as banking and financial services, healthcare, retail, manufacturing, and communications.

This report explores each phase of the internship and provides insights into how the training combined theoretical concepts with practical exposure to real-world full-stack development technologies.

1.1 Academic Relevance and Learning Objectives

The internship was designed to integrate academic learning with practical industry exposure in full-stack development, backend engineering, microservices, and DevOps practices. It acted as an application layer for subjects related to programming, databases, software engineering, web technologies, and cloud computing.

The internship's objectives included:

- Reinforcing Core Java programming and Object-Oriented Programming principles
- Building strong foundations in Data Structures, Algorithms, and Design Patterns
- Learning backend development using Spring Core, Spring Boot 3, and Spring REST
- Mastering ORM and data persistence using Spring Data JPA and Hibernate
- Implementing Test-Driven Development (TDD) using JUnit 5 and Mockito
- Learning microservices architecture using Spring Cloud and containerization using Docker
- Gaining frontend expertise in Node.js, TypeScript, Bootstrap 5, and React (v18)
- Understanding code quality standards and static analysis using SonarQube
- Exploring cloud computing and DevOps using Amazon Web Services (AWS)
- Exploring AI-assisted development using GitHub Copilot
- Improving analytical thinking, problem-solving, and agile collaboration skills

The internship helped transition from academic learning to an industry-ready approach focused on scalable application development, agile delivery, and modern software practices.

1.2 Structure and Phases of the Internship

The internship was organized in a structured and modular format following the DevLed Skilling methodology, where knowledge and skills were developed progressively across five agile sprints. The phases included:

- Bootcamp & Induction (Feb 24 – Mar 4, 2026): Onboarding, company culture, policies, and workplace orientation
- Sprint 0 – Refresher (HTML5, CSS3, JavaScript, ANSI SQL with MySQL, Core Java up to Java 25)
- Sprint 1 – Project Environment Setup, Backend and Data Foundations (Agile, Jira, Git, DSA, Design Patterns, Spring Core, Maven, Spring Data JPA, Hibernate)
- Sprint 2 – API Development and Testing (Spring REST with Spring Boot 3, Backend Debugging, JUnit, Mockito, SLF4J, Lombok)
- Sprint 3 – Microservices and Containerization (Microservices with Spring Boot 3 and Spring Cloud, Docker, GitHub Copilot)
- Sprint 4 – Advanced UI Development (Node.js, TypeScript, Bootstrap 5, React v18, Front-end Debugging)
- Sprint 5 – DevOps, Cloud, and Code Quality (Code Quality Standards, SonarQube, AWS Cloud and DevOps)

- Behavioural Training: Communication, Email Etiquette, Group Discussion, Critical Thinking, and Interview Skills
- Capstone Project (Integrated Development Project): A team-based full-stack application built across all sprints
- Interim Evaluation, Final Evaluation, and HackerRank Final Assessment

Each phase built upon previous learning, ensuring a comprehensive, end-to-end understanding of full-stack application development in line with Cognizant's day-one billability objective.

1.3 Tools, Technologies, and Platforms

The internship emphasized practical exposure to tools and technologies widely used in modern enterprise software development. The tools and technologies included:

- Core Java (up to Java 25): Application programming and OOP
- Spring Framework, Spring Boot 3, Spring Cloud: Backend and microservices development
- Spring Data JPA & Hibernate: ORM and database persistence
- MySQL Community Server 8.4 & MySQL Workbench: Relational database management
- Maven: Build automation and dependency management
- JUnit 5 & Mockito: Unit testing and Test-Driven Development
- SLF4J, Logback & Lombok: Logging and boilerplate reduction
- Node.js, npm, TypeScript: JavaScript runtime and typed front-end development
- Bootstrap 5 & React (v18): Responsive UI and SPA development
- Docker: Containerization and consistent deployment environments
- Git, GitHub, and AWS CodeCommit: Version control and source code management
- Jira: Agile project tracking and Scrum management
- Postman & Swagger / Springdoc: API testing and documentation
- SonarQube: Continuous code quality and static analysis
- Amazon Web Services (AWS): EC2, S3, RDS, Elastic Beanstalk, IAM, CodePipeline, CodeBuild, CodeDeploy
- GitHub Copilot: AI-assisted coding and developer productivity
- IntelliJ IDEA, Eclipse, STS, Visual Studio Code: IDEs for development
- Chrome Developer Tools: Front-end debugging and performance analysis
- GenC Learn Platform & HackerRank: Learning management and assessment

These technologies provided a complete picture of how modern enterprise applications are designed, developed, deployed, and maintained in real-world delivery environments.

1.4 Evaluation and Deliverables

Performance during the internship was monitored through structured evaluations and continuous assessments under Cognizant's Performance Health Status (PHS) framework. The evaluation framework consisted of:

- Self-paced learning module completion on the GenC Learn platform
- Mandatory hands-on exercises and Integrated Capability Tests for each sprint
- Active participation in agile ceremonies (Sprint Planning, Daily Stand-ups, Sprint Reviews, Retrospectives)
- Interim Evaluation (after Sprint 2, around Week 9): One-on-one technical and project review by BU SMEs with RAG (Red-Amber-Green) grading
- Final Evaluation (end of Sprint 5, Week 15): One-on-one technical interview and Capstone project walkthrough
- HackerRank Mock Assessment and Final Assessment for benchmarking technical proficiency
- Trainer feedback, classroom session attendance, and behavioural assessments

These evaluations ensured consistent progress and helped in improving both technical and analytical skills. They also contributed to developing soft skills such as communication, teamwork, time management, and stakeholder interaction. A Green PHS in the Final Evaluation was the gating criteria for deployment readiness.

1.5 Summary of Learning Outcomes

By the end of the internship, I had achieved the following:

- Strengthened Core Java, OOP, DSA, and Design Patterns fundamentals
- Developed enterprise-grade backend applications using Spring Boot 3, Spring Data JPA, and Hibernate
- Built RESTful APIs and applied TDD using JUnit 5 and Mockito
- Designed and deployed microservices using Spring Cloud and Docker
- Built modern Single Page Applications using React (v18), TypeScript, and Bootstrap 5
- Acquired proficiency in code quality monitoring using SonarQube
- Gained exposure to cloud deployment and CI/CD pipelines on AWS

- Practised agile delivery through sprint cycles, Jira, and Scrum ceremonies
- Explored AI-assisted coding using GitHub Copilot
- Improved problem-solving, communication, and collaborative software engineering skills

This internship served as a strong foundation for building a career in full-stack development and modern software engineering, preparing me for real-world client engagements at Cognizant.

2 Overview of the Internship Providing Organization

This section provides a structured overview of Cognizant Technology Solutions, the organization that provided the internship. Cognizant is a global leader in IT services, consulting, and digital transformation, offering end-to-end solutions in cloud computing, artificial intelligence, data analytics, application development, cybersecurity, and enterprise technologies.

The internship program provided exposure to Cognizant's training methodology — the DevLed Skilling Program — its delivery-focused learning approach, and its industry-oriented skill development framework. It emphasized practical learning, continuous evaluation, and alignment with modern technology trends and customer-centric delivery practices.

2.1 Company Profile

Cognizant Technology Solutions Corporation is a leading multinational professional services and technology company headquartered in Teaneck, New Jersey, USA, with a strong global presence across more than 35 countries and a workforce of approximately 350,000 professionals worldwide. Cognizant is listed on the NASDAQ-100 and is part of the S&P 500.

The company provides services across the following key areas:

- Application Development and Maintenance (ADM)
- Cloud Computing, Migration, and Digital Transformation
- Artificial Intelligence, Data, and Analytics
- Enterprise Platforms (SAP, Salesforce, Oracle, Microsoft)
- Internet of Things (IoT) and Engineering Services
- Cybersecurity and Risk Management
- Business Process Services and Consulting

Cognizant helps organizations modernize their IT infrastructure, adopt cloud-based and AI-driven solutions, and accelerate digital transformation across industries.

2.2 Vision, Mission, and Core Values

Cognizant's vision is to engineer modern businesses to improve everyday life by helping clients become data-enabled and AI-empowered enterprises that thrive in the digital era.

Its mission focuses on helping clients transform their business and operating models through advanced technologies such as cloud, artificial intelligence, automation, and data engineering, while delivering measurable business outcomes.

The company operates on core values such as:

- Start with the Client – Customer obsession and value delivery
- Raise the Bar – Pursuit of excellence and continuous improvement
- Dare to Innovate – Bold thinking and innovation
- Do the Right Thing – Integrity, accountability, and ethics
- Own It – Ownership and responsibility for outcomes
- One Cognizant – Collaboration, teamwork, and inclusion

These values are deeply reflected in its work culture, training methodology, and client delivery approach.

2.3 Cognizant in India

Cognizant has one of the largest delivery footprints in India, with multiple delivery centres and offices spread across major cities such as:

- Chennai (largest delivery centre and the location of this internship)
- Bangalore
- Hyderabad
- Pune
- Mumbai
- Kolkata
- Coimbatore
- Gurgaon and Noida

These centres play a central role in Cognizant's global service delivery, innovation labs, and large-scale entry-level talent development programs such as GenC.

2.4 The GenC Program and Learning Environment

The GenC (Generation Cognizant) program is Cognizant's flagship campus-to-corporate transformation initiative for engineering graduates. The program provides a structured learning environment that focuses on continuous skill development, agile delivery readiness, and industry alignment. As GenC Programmer Analyst Trainees, interns are exposed to:

- Technical training across full-stack engineering, cloud, and modern delivery practices
- DevLed Skilling – a project-based, agile, delivery-focused training methodology
- Hands-on exercises, real-world simulations, and Integrated Development Projects

- Access to the GenC Learn platform with curated learning paths and Udemy resources
- Continuous assessments, BU SME evaluations, and HackerRank assessments
- Behavioural and communication training (Cognorator, Business Communication, Interview Clinic)
- Cohort mentoring and leadership connect sessions

This approach ensures that GenCs gain both theoretical knowledge and practical skills required for day-one billability on live client projects.

2.5 Summary

Cognizant is a globally recognized technology leader that emphasizes innovation, learning, customer-centricity, and professional growth. Its strong focus on digital transformation, cloud, AI, and modern engineering practices makes it an ideal platform for fresh engineering graduates to gain industry exposure.

The internship at Cognizant Chennai provided valuable insights into real-world technologies and delivery models and helped in building a strong technical foundation, making it a significant step toward a professional career in the IT services industry.

3 About the Training

3.1 Bootcamp and Induction Phase

The Bootcamp and Induction phase, conducted from 24th February 2026 to 4th March 2026, constituted the preliminary phase of the GenC program at Cognizant Chennai and was designed to establish a strong cultural, professional, and operational foundation. The primary objective of this phase was to familiarize incoming GenC Programmer Analyst Trainees with the company's culture, policies, values, and expectations before the start of structured technical training.

The induction commenced with an in-depth introduction to Cognizant as an organization, its global delivery model, business units, and service lines. Sessions on Cognizant's vision, mission, core values, and code of conduct laid the foundation for professional behaviour. Mandatory orientation modules covered the employee handbook, leave and attendance policies, information security policies, the Cognizant Code of Ethics, and anti-harassment policies.

Subsequently, sessions on Cognizant's organizational structure, the GenC program lifecycle, and the role of a Programmer Analyst Trainee provided clarity on career progression and expectations. Mandatory e-learning modules on information security, data privacy, secure coding, and Responsible AI were completed during this phase. Initial workplace setup activities such as ID card issuance, system provisioning, account creation, access to the GenC Learn platform, MyCognizant portal, and other internal tools were also completed during this period.

In addition to organizational induction, significant importance was given to behavioural and communication readiness. The Behavioural Training (BH) Pre-Assessment, Email Etiquette, and Group Discussion modules were initiated during this phase. These modules form part of the broader Behavioural Training scope spanning the entire program and are crucial for effective interaction in a corporate environment.

In conclusion, the Bootcamp and Induction phase successfully established a strong base in organizational, cultural, and interpersonal domains, thereby preparing trainees for the advanced technical training in the ADM Java Full Stack – React (Stage 2 Plus) track that commenced on 5th March 2026.

3.2 Domain Deployment – ADM Java Full Stack React

Following the successful completion of the Bootcamp and Induction phase, deployment into the ADM Java Full Stack – React (Stage 2 Plus) domain was confirmed based on the candidate profile, skill alignment, and business requirements. From 5th March 2026 onwards, structured technical training was undertaken under the GenC ADM track at Cognizant Chennai, focusing on a wide range of industry-relevant technologies, agile practices, and software engineering disciplines. The learning path was designed using Cognizant's DevLed Skilling methodology to provide both conceptual understanding and practical exposure to modern full-stack development and enterprise application ecosystems.

The training was structured across five agile sprints over 14.24 weeks, beginning with a Sprint 0 Refresher covering HTML5, CSS3, JavaScript, ANSI SQL with MySQL, and Core Java up to Java 25. The refresher established foundational competence across web technologies, relational databases, and Java programming.

Sprint 1 focused on Project Environment Setup, Backend, and Data Foundations through modules on Agile Methodology, Jira Basics, Git Basics, Data Structures and Algorithms, Design Patterns and Principles, Spring Core with Maven, and Spring Data JPA with Spring Boot and Hibernate.

Sprint 2 was dedicated to API Development and Testing, covering Spring REST using Spring Boot 3, backend application debugging, Test-Driven Development using JUnit 5 and Mockito, and structured logging with SLF4J and Lombok. An Interim Evaluation was conducted at the end of this sprint.

Sprint 3 covered Microservices and Containerization through Microservices with Spring Boot 3 and Spring Cloud, Docker basics, and GitHub Copilot for AI-assisted development.

Sprint 4 focused on Advanced UI Development with Node.js fundamentals, TypeScript basics, Bootstrap 5, React (v18), and front-end application debugging.

Sprint 5 concluded the technical training with DevOps, Cloud, and Code Quality, covering Code Quality Standards, SonarQube, and Cloud and DevOps using AWS, followed by the Final Evaluation and HackerRank Mock & Final Assessment.

Throughout the training, agile ceremonies — Sprint Planning, Daily Stand-ups, Sprint Reviews, and Retrospectives — were conducted alongside continuous classroom sessions delivered by technical Subject Matter Experts. A capstone Integrated Development Project (IDP) ran in parallel from Sprint 1 through Sprint 5, requiring teams to design, build, test, and deliver a complete full-stack application demonstrating end-to-end mastery of the curriculum.

3.2.1 Agile Methodology, Jira, and Git

During Sprint 1 of the Stage 2 Plus training, a comprehensive understanding of Agile and Scrum methodologies was developed, focusing on both theoretical concepts and practical implementation through Cognizant's DevLed sprint cycles.

Agile is a modern software development approach that emphasizes iterative progress, collaboration, flexibility, and continuous feedback. Unlike traditional waterfall models, Agile promotes incremental delivery through short development cycles known as sprints. The Agile Manifesto and its four values and twelve principles were studied in depth, along with their comparison to the waterfall methodology.

The Scrum framework was studied through its three roles — Product Owner, Scrum Master, and Development Team — along with its key ceremonies (Sprint Planning, Daily Stand-up, Sprint Review, Sprint Retrospective) and artifacts (Product Backlog, Sprint Backlog, Increment). Techniques such as Story Points, Planning Poker, writing effective User Stories with acceptance criteria, and tracking progress through Velocity and Burndown Charts were practised throughout the program.

Jira was used as the primary Agile project management tool. Hands-on exercises covered creating projects, managing issues (Story, Task, Bug, Epic), navigating Scrum boards, planning and executing sprints, configuring workflows, and using built-in reports such as Burndown, Velocity, and Cumulative Flow. Embedded AI features such as Atlassian Intelligence and Rovo were also explored for AI-driven backlog refinement and automation.

Git was used as the distributed version control system across the entire training. Concepts of version control fundamentals, working directory, staging area, repository, branching, merging, conflict resolution, and remote collaboration were practised. Common commands such as `git init`, `git clone`, `git add`, `git commit`, `git status`, `git log`, `git push`, `git pull`, and `git merge` were applied in real PoD-based collaborative workflows. Branching strategies including Feature Branching, Release Branching, and Git Flow were studied along with Forking and Pull Request based workflows on GitHub.

Overall, this module provided a strong foundation in Agile, Scrum, Jira, and Git, combining theoretical knowledge with the actual sprint cadence followed throughout the entire training program. It highlighted the significance of adaptability, teamwork, ownership, and continuous improvement in modern software delivery.

3.2.2 Data Structures and Algorithms

As part of Sprint 1, a focused module on Data Structures and Algorithms (DSA) was undertaken to strengthen problem-solving and computational thinking. The module emphasised the importance of DSA in software engineering, time and space complexity analysis, and Big O notation for performance evaluation.

Core data structures were studied and implemented in Java, including arrays and strings (with operations like reversal, palindrome checking, and anagram detection), singly and doubly linked lists, circular linked lists, stacks (with applications such as expression evaluation), queues (simple, circular, and priority), trees, and graphs.

Algorithmic techniques such as recursion, backtracking (N-Queens, Sudoku Solver), tail recursion optimization, and classical sorting algorithms (Bubble, Selection, Insertion, Merge, Quick) along with searching algorithms (Linear Search, Binary Search) were practised. Advanced techniques such as Greedy strategies and Dynamic Programming (memoization and tabulation) were applied to problems including the knapsack problem and the Longest Common Subsequence.

This module strengthened logical thinking, coding fluency, and the ability to design optimized algorithms — essential skills for the HackerRank assessments and real client deliverables.

3.2.3 Design Patterns and Principles

As a continuation of Sprint 1, the Design Patterns and Principles module focused on writing clean, maintainable, and extensible Java code. Software design principles such as SOLID (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion), DRY, KISS, YAGNI, and the trade-offs between Coupling and Cohesion were studied with Java examples.

Object-oriented design concepts including encapsulation, inheritance, polymorphism, abstraction, and the preference of composition over inheritance were reinforced through practical modelling exercises. Classical Gang of Four design patterns were studied across three categories:

- Creational Patterns: Singleton (eager and lazy), Factory, Abstract Factory, Builder, and Prototype
- Structural Patterns: Adapter, Decorator, Composite, Proxy, and Facade
- Behavioural Patterns: Strategy, Observer, Command, Template Method, State, and Chain of Responsibility

Refactoring techniques were practised to identify code smells and remove anti-patterns such as God Object, Spaghetti Code, and Golden Hammer. Real-world examples in Spring Boot and UI logic helped contextualise where each pattern is applied in enterprise applications.

3.2.4 Spring Core and Maven

The Spring Core and Maven module formed the backbone of backend training in Sprint 1. The Spring Framework was introduced through its core concepts of Inversion of Control (IoC) and Dependency Injection (DI), along with an overview of Spring modules including Core, AOP, Data Access, ORM, and MVC.

Maven was used as the build automation and dependency management tool. Hands-on exercises included creating Maven projects, configuring the pom.xml file, managing dependencies, and executing Maven lifecycle goals such as compile, test, package, and install.

The Spring IoC container was configured using both XML and annotation-based approaches. Bean configuration using `@Component`, `@Service`, `@Repository`, `@Controller`, `@Configuration`, and `@Bean` was practised along with component scanning. Dependency Injection was applied through constructor injection, setter injection, and autowiring with `@Autowired`, `@Qualifier`, `@Resource`, and `@Inject`.

Spring AOP was studied through aspects, advices, pointcuts, and joinpoints, demonstrating cross-cutting concerns such as logging and transaction management. An introductory overview of Spring Boot, auto-configuration, and reactive programming with Spring WebFlux (Mono, Flux) was also covered.

3.2.5 Spring Data JPA with Spring Boot and Hibernate

The Spring Data JPA module provided in-depth training on data persistence in modern Spring Boot applications. The relationship between JPA, Hibernate, and Spring Data JPA was studied to understand the layered abstraction over JDBC.

Hands-on exercises included creating Spring Boot projects with Spring Data JPA dependencies, configuring application.properties for MySQL connectivity, defining entities using annotations such as `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@OneToOne`, `@OneToMany`, `@ManyToOne`, and `@ManyToMany`, and mapping entity relationships.

Spring Data Repositories were created by extending `JpaRepository` and `CrudRepository` to implement CRUD operations with minimal boilerplate. Derived query methods, custom queries

using `@Query` with JPQL and native SQL, and named queries were practised. Pagination and sorting using `Page` and `Pageable` were applied to large data sets.

Auditing was enabled using `@CreatedBy`, `@LastModifiedBy`, `@CreatedDate`, and `@LastModifiedDate`. Spring Data JPA Projections (interface-based and class-based) were used to retrieve specific data subsets efficiently. Hibernate-specific features such as dialect configuration, lazy/eager loading strategies, and batch processing were also explored.

3.2.6 Spring REST using Spring Boot 3

Sprint 2 commenced with the Spring REST module, focusing on building production-grade RESTful web services using Spring Boot 3. The fundamentals of REST architecture, HTTP methods (GET, POST, PUT, DELETE, PATCH), status codes, and statelessness were studied along with new features in Spring Boot 3.

Hands-on exercises covered creating REST controllers with `@RestController`, defining request mappings with `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping`, and handling path variables and query parameters using `@PathVariable` and `@RequestParam`. Request and response bodies were processed using `@RequestBody` and `ResponseEntity`, and global exception handling was implemented using `@ControllerAdvice` and `@ExceptionHandler`.

RESTful resource representation was implemented using Data Transfer Objects (DTOs), with custom JSON serialization and deserialization using Jackson. Concepts such as HATEOAS, content negotiation, multiple media types (JSON, XML), and API versioning were studied.

Security aspects including securing endpoints with Spring Security, basic authentication, JWT-based token authentication, role-based authorization, and CORS handling were introduced. Spring Boot Actuator was used for monitoring and exposing operational endpoints. APIs were tested using Postman and documented using Swagger / Springdoc OpenAPI, providing automatically generated and interactive API documentation.

3.2.7 Backend Application Debugging

The Application Debugging – Backend module trained me to systematically diagnose and resolve runtime issues in Java backend applications. Debugging fundamentals such as setting breakpoints, stepping through code, watching variables, and inspecting call stacks were practised in IntelliJ IDEA and Eclipse / STS.

Specific debugging scenarios were covered including REST endpoint debugging (request mapping, URL handling, request and response inspection), authentication and authorization issues, and asynchronous code debugging involving thread inspection and concurrency analysis. Reading and analysing log files and stack traces formed an important part of the workflow.

3.2.8 Test-Driven Development using JUnit 5 and Mockito

Test-Driven Development (TDD) was introduced through the Red-Green-Refactor cycle. Hands-on exercises with JUnit 5 covered writing and running unit tests, using annotations such as `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, and `@AfterAll`, and writing assertions to validate expected outcomes.

Advanced JUnit features including parameterized tests, test suites, exception testing using `assertThrows`, timeout testing, and execution order control were practised. Mockito was used for mocking and stubbing dependencies, verifying interactions, argument matching, and handling void methods.

Spring Boot specific testing was covered through `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `MockMvc` for controller testing, and mocking external services and databases. This module instilled a strong testing discipline aligned with the test-first principle promoted in agile delivery.

3.2.9 SLF4J and Lombok

The SLF4J and Lombok module covered structured logging and boilerplate code reduction in Java. SLF4J was studied as a logging facade, with Logback configured as the underlying implementation. Logging levels (TRACE, DEBUG, INFO, WARN, ERROR), structured logging best practices, and exception logging with contextual data were practised.

Default Spring Boot logging was customised through `application.properties` and `logback.xml`. Lombok was used to reduce boilerplate code through annotations such as `@Getter`, `@Setter`, `@ToString`, `@EqualsAndHashCode`, `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`, `@Builder`, `@Slf4j`, and `@Value`. IDE plugin setup for IntelliJ IDEA and Eclipse was performed to ensure smooth Lombok support.

3.2.10 Microservices with Spring Boot 3 and Spring Cloud

Sprint 3 commenced with a deep dive into microservices architecture (MSA). The differences between monolithic and microservices architectures were studied along with the advantages, challenges, and characteristics of microservices.

Spring Cloud components were applied to implement enterprise microservices patterns:

- Service discovery and registration using Spring Cloud Netflix Eureka
- Centralised externalised configuration using Spring Cloud Config
- Inter-service communication using Spring Cloud OpenFeign
- API Gateway and edge services using Spring Cloud Gateway with routing, filtering, and load balancing
- Circuit breaker, fallback, and retry patterns using Resilience4j / Spring Cloud Circuit Breaker
- Centralised authentication and authorization using OAuth 2.1 / OIDC and JWT tokens
- Monitoring with Spring Boot Actuator integrated with Prometheus and Grafana

Security best practices such as Role-Based Access Control (RBAC), securing inter-service communication, and protecting sensitive data in distributed systems were also studied. Inter-service messaging and event-driven communication using brokers such as Kafka were introduced as supporting concepts.

3.2.11 Docker and Containerization

Docker was introduced as the primary containerization technology used to package and deploy applications consistently. The fundamentals of containers, container architecture, Docker images, Docker registries, and the differences between containers and virtual machines were studied.

Practical exercises included writing Dockerfiles using FROM, COPY, RUN, ENTRYPOINT, and EXPOSE instructions, building and tagging Docker images, running containers, port mapping, volume mounting, and container lifecycle management. Hosting an Nginx-based application via the command line and Dockerfile, hosting a MySQL database with schema initialization, and dockerizing a Spring Boot REST microservice were all completed as hands-on labs.

Docker Compose was used to orchestrate multi-container applications, defining services in docker-compose.yml, using depends_on and links to manage dependencies between the REST API service and the MySQL container, and configuring environment variables and persistent volumes.

3.2.12 GitHub Copilot – AI-Assisted Development

GitHub Copilot was introduced as an AI-powered coding companion to enhance developer productivity. Topics covered included prompt engineering best practices, IDE setup and configuration, and core Copilot capabilities such as code suggestions, function and boilerplate

generation, comment and documentation generation, code refactoring, and unit test case generation.

Equally important were sessions on responsible AI usage — understanding risks of AI-generated code, licensing and attribution concerns, data privacy, organisational usage policies, and best practices such as reviewing AI-generated code, combining Copilot with manual coding, using it as a learning aid for new languages, and avoiding over-reliance. This was reinforced by Cognizant's Responsible AI training modules.

3.2.13 Node.js Fundamentals

Sprint 4 began with Node.js fundamentals to support modern frontend tooling and full-stack JavaScript development. Topics covered the role of Node.js in web development, its single-threaded non-blocking I/O architecture, the module system using `require` / `import`, and exporting and importing modules.

Core Node.js modules were practised, including the File System (`fs`) module for asynchronous file reading and writing using `readFile` and `writeFile`, and the HTTP module for creating basic HTTP servers and handling requests and responses. Asynchronous programming was covered through callbacks, Promises, `.then()` chaining, and `async/await` syntax. `npm` was used extensively for dependency management.

3.2.14 TypeScript Basics

TypeScript was introduced as a strongly typed superset of JavaScript that compiles to plain JavaScript. Topics covered installation and the TypeScript compiler (`tsc`), `tsconfig.json` configuration, primitive types (number, string, boolean), special types (any, void, null, undefined), arrays, and tuples.

Type annotations, type inference, and type assertions were practised. Functions were enhanced with parameter and return types, optional and default parameters, and typed arrow functions. Interfaces, optional and readonly properties, classes with public, private, and protected access modifiers, basic inheritance, union types, literal types, and type narrowing were all covered with hands-on exercises.

3.2.15 Bootstrap 5

Bootstrap 5 was used to build responsive, mobile-first user interfaces. The Bootstrap framework, file structure, and setup were covered along with the responsive grid system based on rows, columns, breakpoints, and flexbox utilities.

Bootstrap components such as typography, forms, buttons, navbars, navigation, cards, and media objects were used to build polished UIs. Utility classes for spacing, colours, backgrounds, display, visibility, borders, shadows, rounded corners, and positioning were practised. Advanced features such as Bootstrap Icons, JavaScript plugins, and Sass-based customization were also explored.

3.2.16 React (v18)

React (v18) was the centrepiece of frontend training in Sprint 4. The training began with setting up the development environment using Node.js, npm, and create-react-app, and exploring the project structure of a React application. New v18 features such as automatic batching and concurrent rendering were introduced.

JSX syntax, embedding expressions, conditional rendering using ternary and short-circuit operators, and rendering elements with ReactDOM.createRoot were practised. Functional components, props, prop types, default props, parent-to-child data flow, and component composition were extensively used to build reusable UIs.

State management within components was implemented using the useState hook. Side effects and lifecycle behaviour were managed using useEffect with appropriate dependency arrays and cleanup functions. The useRef hook was used for DOM manipulation and persistent values across renders.

React Router was used to implement client-side navigation through BrowserRouter, Route, and Link, with nested routes, route parameters, programmatic navigation using useNavigate, and accessing parameters using useParams. Controlled forms were built with state, integrated with libraries such as Formik or React Hook Form for validation.

Application-wide state management was approached using the Context API (createContext, Provider, useContext) and through Redux with React-Redux, including stores, actions, reducers, useSelector, and useDispatch. Asynchronous operations and data fetching were performed using the fetch API with proper error handling, loading states, and an introduction to React Query for caching.

Testing in React was practised using Jest for unit tests, React Testing Library for component and interaction testing, and a basic introduction to E2E testing with Cypress. The capstone project's frontend was built end-to-end using React, integrating with the Spring Boot REST APIs developed in earlier sprints.

3.2.17 Front-End Application Debugging

Front-end debugging was practised using Chrome DevTools and Visual Studio Code. Activities included launching React applications, inspecting elements and the DOM, examining the Network tab, debugging JavaScript using breakpoints, watches, and the call stack, and analysing the React Components and Profiler tabs of React DevTools.

Performance debugging covered analysing rendering behaviour, memory usage, and network bottlenecks. The Visual Studio Code debugger was attached to the running application to set breakpoints in TypeScript / JSX files, watch state and props, and trace state management flows in Redux and Context API.

3.2.18 Code Quality Standards and SonarQube

Sprint 5 began with industry-standard code quality practices. Topics covered clean code principles, naming conventions, modularity, code formatting, and common quality metrics such as cyclomatic complexity, code duplication, and test coverage.

SonarQube was integrated into the build process using Maven, with the Sonar architecture (client, server, and database) studied in detail. Maven Sonar goals were executed for static code analysis, and SonarQube dashboards were used to identify duplicate code, high cyclomatic complexity, code smells, lack of unit tests, deviation from coding standards, potential bugs, and insufficient comments. Sonar logs were interpreted and recommended fixes were applied to improve code health continuously — embodying the "Clean as You Code" philosophy.

3.2.19 Cloud and DevOps using AWS

The Cloud and DevOps module introduced cloud computing fundamentals — the differences between traditional IT, virtualization, Service-Oriented Architecture, and cloud, along with the pros and cons of public, private, hybrid, and community cloud deployment models, and the IaaS, PaaS, and SaaS service models.

Amazon Web Services (AWS) was used as the primary cloud platform. Topics covered included:

- AWS global infrastructure, account creation, and overview of services
- Compute: Amazon EC2 (instance launch, AMIs, security groups, key pairs) and Amazon ECS (Docker-based containers)
- Storage: Amazon S3 (buckets, storage classes, object operations) and Amazon EBS (volumes and snapshots)

- Networking: Amazon VPC (subnets, route tables, security groups, peering), Application Load Balancer (ALB), Network Load Balancer (NLB)
- Databases: Amazon RDS (multi-AZ deployments) and Amazon DynamoDB (NoSQL basics)
- Identity and Access Management: IAM users, groups, roles, policies, and Multi-Factor Authentication
- Serverless: AWS Lambda functions and AWS API Gateway
- Application deployment using AWS Elastic Beanstalk

AWS DevOps services were used to implement CI/CD pipelines:

- AWS CodeCommit for source control and version management
- AWS CodeBuild for compilation, testing, and artifact generation using buildspec.yml
- AWS CodeDeploy for automated deployments using AppSpec files and deployment hooks, with rollback support
- AWS CodePipeline for orchestrating end-to-end CI/CD with monitoring and troubleshooting

This module concluded the technical curriculum by tying together development, code quality, containerization, and cloud deployment into a coherent end-to-end software delivery pipeline.

3.3 Behavioural Training and Soft Skills

Running in parallel with the technical curriculum, the Behavioural Training (BH) track was conducted across all eight weeks of the program. Modules included Email Etiquette, Group Discussion, Problem Solving and Critical Thinking, Debate, Conference Call communication, Tenses and Subject-Verb Agreement, Storytelling, Email and Chat coaching, Article and Basic Grammar, In a Nutshell sessions, Presentation Skills, Client Interfacing Skills, and Project Simulation.

An Interview Clinic and Mock Interview were conducted before the Interim Evaluation, and Cohort Mentor Sessions and Leadership Connects were scheduled throughout the program. The BH Pre-Assessment and Post-Assessment measured the development of communication and behavioural readiness over the duration of the program.

3.4 Capstone Integrated Development Project (IDP)

The Capstone Integrated Development Project ran in parallel from Sprint 1 through Sprint 5 as the final integration of all learning. As part of an agile pod, I contributed to the design and development of a complete full-stack application that included:

- A Spring Boot 3 backend with layered architecture (Controller, Service, Repository)
- Spring Data JPA and Hibernate persistence with MySQL
- RESTful APIs documented with Swagger / Springdoc OpenAPI
- Unit testing using JUnit 5 and Mockito with TDD discipline
- Decomposition into microservices using Spring Cloud (Eureka, Config, Gateway, OpenFeign)
- Containerization of services and the database using Docker and Docker Compose
- A React (v18) Single Page Application with Bootstrap 5 and TypeScript
- Static code analysis using SonarQube and continuous code health monitoring
- Local deployment, with cloud deployment exercises on AWS EC2, S3, and RDS

The project was managed in Jira using Scrum, with all agile ceremonies (Sprint Planning, Daily Stand-ups, Sprint Reviews, Retrospectives) followed strictly. The Final Sprint Evaluation involved a one-on-one technical and project walkthrough with BU Subject Matter Experts, where code quality, architecture decisions, UI/UX, database schema design, API design and validation, and the team's ability to justify architectural choices were assessed.

3.5 Evaluation and Assessment

The training included two formal one-on-one evaluations conducted by BU Subject Matter Experts and a final HackerRank assessment, all aligned to Cognizant's Performance Health Status (PHS) framework with RAG (Red-Amber-Green) grading.

3.5.1 Interim Evaluation

The Interim Evaluation was conducted after Sprint 2 (around Week 9) and served as the first major formal gatekeeper assessment of the program. It validated the foundational competencies built up to that point, spanning the Bootcamp, Sprint 0 Refresher, and Sprints 1–2.

The evaluation focused on:

- Technical competence: Core Java, OOP, DSA, Design Patterns, Agile, Git, Spring Core, Maven, Spring Data JPA with Hibernate, Spring REST with Spring Boot 3, JUnit, Mockito, SLF4J, and Lombok

- Project contribution: Hands-on completion, collaboration in the agile pod, and progress on the initial setup of the IDP
- Format: One-on-one technical interview and project review with a BU SME
- Outcome: RAG grading — Green (on track), Amber (improvement areas identified), or Red (significant gaps)

Passing the Interim Evaluation was a prerequisite for progressing into microservices, frontend, and DevOps modules in Sprints 3 to 5.

3.5.2 Final Evaluation and HackerRank Assessment

The Final Evaluation was conducted at the end of Sprint 5 (Week 15) and represented the definitive validation of the entire curriculum. It was the official gateway to project-readiness within Cognizant's delivery organization.

The Final Evaluation focused on:

- Technical competence across the full stack: Microservices with Spring Cloud, Docker, React (v18), TypeScript, Bootstrap 5, Code Quality, SonarQube, AWS Cloud, and DevOps
- Capstone project delivery: architecture, implementation, testing, and deployment
- Format: One-on-one technical interview and detailed project walkthrough
- Outcome: RAG grading — a Green PHS being the gating criteria for deployment readiness

In addition to the SME evaluation, the program concluded with a HackerRank Mock Assessment followed by the HackerRank Final Assessment, which independently benchmarked coding proficiency across Java, SQL, REST, and front-end concepts.

Following these evaluations, a final revision and Sprint Implementation phase reinforced any technical gaps identified, with self-managing Scrum teams, Sprint Planning, Daily Scrums, development cycles, and a concluding Sprint Demo to a panel of Lead Architects. Successful completion of the entire evaluation suite signified day-one billability — readiness to contribute to live client engagements as a Cognizant associate.

4 Conclusion

The internship at Cognizant Technology Solutions, Chennai, under the GenC ADM Java Full Stack – React (Stage 2 Plus) track proved to be a highly enriching and multidimensional learning experience. It provided me with an opportunity to enhance my technical knowledge while gaining first-hand exposure to the real-world working environment of a global IT services and consulting organization. Cognizant is a leading technology partner that helps businesses transform through cloud, AI, data, and digital solutions, making this internship a valuable platform for industry-oriented learning.

The structured training across various domains — including Core Java, Object-Oriented Programming, Data Structures and Algorithms, Design Patterns, Spring Core, Spring Boot 3, Spring Data JPA, Hibernate, Spring REST, JUnit 5, Mockito, SLF4J, Lombok, Microservices with Spring Cloud, Docker, Node.js, TypeScript, Bootstrap 5, React (v18), SonarQube, AWS Cloud and DevOps, GitHub Copilot, Jira, and Git — helped in building a strong foundation in full-stack application development and modern software engineering practices. The training modules were designed to provide both conceptual clarity and practical understanding, enabling me to grasp how enterprise applications are designed, developed, tested, and deployed in real-world delivery environments.

In addition to technical learning, the internship contributed significantly to my overall professional development. The DevLed Skilling methodology emphasised analytical thinking, problem-solving, agile collaboration, and adaptability, which are essential skills in the IT services industry. Exposure to project-based learning through the capstone Integrated Development Project, daily stand-ups, sprint reviews, retrospectives, and continuous assessments enhanced my understanding of how applications are built, tested, optimized, and delivered to stakeholders.

Another important aspect of the internship was understanding the integration of development, code quality, and operations through DevOps practices. Learning about CI/CD pipelines on AWS using CodeCommit, CodeBuild, CodeDeploy, and CodePipeline, along with code quality enforcement using SonarQube, version control using Git, and AI-assisted development using GitHub Copilot, strengthened my awareness of modern software delivery workflows and automation.

The behavioural training track and the cohort mentoring sessions further developed my communication skills, professional etiquette, client interfacing capability, and confidence in stakeholder interactions. The Interim and Final Evaluations conducted by BU Subject Matter Experts, along with the HackerRank Final Assessment, validated my technical and project readiness against Cognizant's delivery standards.

The internship has now been successfully completed and has played a crucial role in shaping my technical and professional skills. It has prepared me to take on future challenges in the field of full-stack development, microservices, and software engineering as a GenC Programmer Analyst Trainee at Cognizant.

In conclusion, this internship has been a transformative experience that strengthened my knowledge in application development, improved my problem-solving abilities, and provided valuable exposure to industry practices. While this report summarizes my learning and experience, certain internal details have been omitted in accordance with professional and organizational confidentiality guidelines. I now move forward with confidence, equipped with the skills, mindset, and knowledge required to build a successful career in software development and modern application technologies.